Zuse 3 – Im Inneren des Prozessors

Konzept des dritten Bandes der Zuse-Reihe: Unterbrechungen, Zeitgeber, Ports, Von-Neumann und Harvard, Speicherhierarchie, Fließband, mehrere Kerne, vom Text zum Befehl und der Prozessorkern Zuse-K für 14 bis 16 Jahre

Dokument	Konzept
Version	0.1
Status	ENTWURF
Autor	Daniel Zeimentz · DC Software Engineering
Datum	14.11.2026
Bezug	Konzept Zuse v0.3 (Reihe, Abschnitt 7.3); Konzept Zuse 2 v0.1; Konzept MCU Explorer v0.2 (Abschnitt 4, Arbeitsteilung); ZUSE NPL Sprachkern v0.1 (Schreibweise der Schlüsselwörter); DC Learning Ecosystem v1.1, Design Standard v1.0
Zielplattformen	Windows zuerst, dann Android-Tablet, iPadOS, macOS, Linux
Entscheidungen	14.11.2026 (Inhaber): Zuschnitt der Skizze beibehalten und erweitert um »Vom Text zum Befehl«, »Cache und Speicherhierarchie« und »Mehrere Kerne«

Inhalt

1. Zusammenfassung **3**

2. Ausgangslage **3**

2.1 Was Zuse 2 mitgibt . . . 3

2.2 Lehrplanbezug . . . 3

2.3 Arbeitsteilung mit dem MCU Explorer . . . 4

3. Ziele und Nicht-Ziele **4**

3.1 Ziele . . . 4

3.2 Nicht-Ziele . . . 4

4. Lösungskonzept **5**

4.1 Der Zuse-K . . . 5

4.2 Messleiste und Vergleich . . . 5

4.3 Darstellungen . . . 5

4.4 Vom Text zum Befehl: Zuse-Text . . . 5

5. Lernpfad **6**

6. Technische Architektur **8**

7. Datenschutz, Kindersicherheit, Barrierefreiheit **8**

8. Anforderungen **8**

9. Risiken und Maßnahmen **9**

10. Umsetzungsplan **9**

11. Offene Entscheidungen **9**

12. Fazit und Empfehlung **10**

1. Zusammenfassung

Zuse 2 endet mit einem selbst gebauten 8-Bit-Rechner, der in einer Schleife ständig seine Tasten abfragt, und mit der Frage: **Wie reagiert ein Rechner auf die Welt, ohne ständig nachzufragen?** Zuse 3 beantwortet sie und öffnet den Blick auf die Bauweise heutiger Prozessoren. Aus dem Zuse-8 wird der **Zuse-K**, ein kleiner Prozessorkern mit Unterbrechungen, Zeitgeber und Ports, umschaltbar zwischen Von-Neumann- und Harvard-Bauweise, mit Zwischenspeicher (Cache), Fließband und – im Vergleich – zwei Kernen an einem Speicher.

Zum Schluss schreiben die Jugendlichen Programme in einer kleinen eigenen Sprache (**Zuse-Text**, Arbeitsname). Zetti übersetzt sie Schritt für Schritt in Zuse-K-Befehle und zeigt so, was ein Übersetzer (Compiler) tut. Die letzte Stufe zeigt die Register eines echten Mikrocontrollers in der Zuse-Darstellung und führt zum MCU Explorer.

Zuse 3 baut, wie Zuse 1 und 2, **alles selbst und bleibt rein digital**. Unterbrechungen und Zeitgeber entstehen aus Gattern und Registern. Ihre Benutzung an einem echten Chip in C/C++ mit elektrischer Umgebung gehört zum MCU Explorer.

Umfang: 10 Lernstufen mit 38 Experimenten, geschätzt 140–185 PT. **Empfehlung:** Start nach Zuse 2. Vorher ein Machbarkeitstest: Zuse-K auf Registertransfer-Ebene, Fließband- und Cache-Darstellung, zwei Kerne mit reproduzierbarer Reihenfolge, Übersetzer.

2. Ausgangslage

2.1 Was Zuse 2 mitgibt

Aus Zuse 2	Wiederverwendung in Zuse 3
Zuse-8 mit Mikroprogramm, Stapel, IN/OUT, Pixelmatrix	Grundlage des Zuse-K; neue Befehle über das Mikroprogramm
Assembler mit Marken, Haltepunkten, Intel HEX	unverändert; erweitert um Unterbrechungsroutinen und Datenbereiche
Bus mit Zustand Z, Speicher 256 × 8, Adressbus	Ports als Speicheradressen, zweiter Speicherbus (Harvard)
Zähler, Vergleicher, Register aus Zuse 1	Zeitgeber-Baustein
Zeitdiagramm (Logikanalysator)	Fließbanddiagramm, Busbelegung zweier Kerne

2.2 Lehrplanbezug

Informatik Ende Sek. I und Sek. II: Rechnerarchitektur (Von-Neumann, Harvard), Speicherhierarchie, Unterbrechungen, Nebenläufigkeit (Wettlaufsituation, wechselseitiger Ausschluss), Übersetzung von Hochsprache in Maschinensprache, Leistung von Prozessoren. In der Berufsausbildung (IT-Berufe, Elektronik) Grundlagen der Mikroprozessortechnik.

2.3 Arbeitsteilung mit dem MCU Explorer

Thema	Zuse 3 (bauen, verstehen)	MCU Explorer (benutzen)
Unterbrechungen	Unterbrechungslogik aus Gattern; Rücksprung über den Stapel	`attachInterrupt`, Vektoren, Prellen
Zeitgeber	aus Zähler und Vergleicher	Timer-Register, `millis()`, PWM
Ein-/Ausgabe	Port als Speicheradresse, Richtungsregister selbst gebaut	GPIO, Pull-up, Pegelschwellen, Strom
Architektur	Von-Neumann/Harvard umschaltbar, Cache, Fließband, zwei Kerne	ATmega328P als Harvard-Rechner ohne Cache
Sprache	Zuse-Text → Zuse-K-Befehle, jeder Schritt sichtbar	C/C++ mit avr-gcc; Disassembler (MCU Explorer 2)
Elektrik	keine	volle Rückwirkung über den Volta-Kern

3. Ziele und Nicht-Ziele

3.1 Ziele

- Verstehen, warum heutige Prozessoren so gebaut sind: Unterbrechung statt Abfrage, Zeitgeber statt Warteschleife, Cache gegen langsamen Speicher, Fließband und mehrere Kerne für mehr Leistung.
- Jedes neue Bauteil wird selbst gebaut und am eigenen Programm **gemessen** (Takte, Treffer, Wartezeiten) – Leistung wird zählbar.
- Übersetzung von Hochsprache als nachvollziehbarer Vorgang statt als Zauberei.
- Saubere Übergabe an den MCU Explorer.

3.2 Nicht-Ziele

- Kein Nachbau eines realen Prozessors oder Befehlssatzes; kein Betriebssystem.
- Keine allgemeine Programmiersprache: Zuse-Text bleibt eine kleine Lehrsprache.
- Keine C/C++-Programmierung, keine Sensorik, keine Elektrik (MCU Explorer).
- Keine Netzwerkfunktionen; der Sprachkern ZUSE NPL (Netzwerk) ist ein eigenes Vorhaben.

4. Lösungskonzept

4.1 Der Zuse-K

Merkmal	Festlegung
Kern	Zuse-8 (Abschnitt 4.3 im Konzept Zuse 2) mit Mikroprogramm
Neue Befehle	EI, DI (Unterbrechungen erlauben/sperren), RETI (Rückkehr aus Unterbrechung), TAS (Prüfen und Setzen, für Sperren)
Unterbrechungen	zwei Quellen (Taste, Zeitgeber) mit fester Rangfolge; Merker je Quelle; Sprung an feste Adressen; Befehlszähler und Merker automatisch auf den Stapel
Zeitgeber	8-Bit-Zähler mit Vorteiler und Vergleichsregister; löst bei Gleichheit eine Unterbrechung aus oder schaltet einen Ausgang um (Ton)
Ports	ab Adresse F0: Datenregister und Richtungsregister je Port; IN/OUT bleiben zum Vergleich
Bauart	umschaltbar: **Von-Neumann** (ein Speicher, ein Bus) oder **Harvard** (Programmspeicher und Datenspeicher, zwei Busse); Konstanten im Programmspeicher
Speicherhierarchie	langsamer Hauptspeicher (einstellbar 1–8 Wartetakte) und Cache mit 4 oder 8 Zeilen, direkt abgebildet, Durchschreiben
Fließband	zuschaltbar, drei Stufen (Holen, Entschlüsseln, Ausführen); Sprünge leeren das Band, Datenkonflikte erzeugen Wartetakte
Zwei Kerne	zwei Zuse-K an einem gemeinsamen Speicher mit Schiedsrichter (Arbiter); Reihenfolge wechselnd oder per Würfel mit sichtbarem Startwert
Zähler	Takte, Befehle, Wartetakte, Cache-Treffer und -Fehlschüsse, Band-Leerungen – sichtbar in einer Messleiste

4.2 Messleiste und Vergleich

Neu gegenüber Zuse 2 ist die **Messleiste**. Jede Bauweise wird mit demselben Programm gemessen. Das Kind sieht zum Beispiel: »Abfrage: 1.240 Takte, davon 1.100 Warten – Unterbrechung: 160 Takte«. Zwei Läufe lassen sich nebeneinander vergleichen. So beruht jede Aussage über »schneller« auf einer eigenen Messung und nicht auf einer Behauptung.

4.3 Darstellungen

- **Fließbanddiagramm:** Zeilen = Befehle, Spalten = Takte; jede Zelle zeigt die Stufe als Buchstabe (H, E, A) und Form. Leerungen und Wartetakte sind als »–« bzw. »warten« beschriftet.
- **Cache-Ansicht:** Tabelle der Zeilen (gültig, Adresskennung, Inhalt). Jeder Zugriff wird als »Treffer ✓« oder »Fehlschuss ✗« markiert und gezählt.
- **Zwei Kerne:** zwei Bedienfelder nebeneinander, dazwischen der gemeinsame Speicher; eine Busleiste zeigt, wer gerade zugreift.
- **Unterbrechung:** Der Programmzähler springt sichtbar (Pfeil in der Speicheransicht). Der Stapel zeigt die gesicherte Rücksprungadresse. Die LED »Unterbrechung« trägt Wort und Symbol.

4.4 Vom Text zum Befehl: Zuse-Text

Zuse-Text (Arbeitsname) ist eine kleine Lehrsprache. Ihre Schlüsselwörter sind wie im Sprachkern ZUSE NPL großgeschrieben und werden in jeder Sprache übersetzt: WENN/DANN/SONST, SOLANGE, ENDE. Programme werden sprachneutral gespeichert, sodass ein Sprachwechsel das Programm nicht ändert.

Umfang (Version 1)	Inhalt
Daten	Variablen zu einem Byte, Zahlen dezimal/binär/hex

Umfang (Version 1)	Inhalt
Ausdrücke	+, −, UND, ODER, Vergleiche (=, ≠, <, >)
Anweisungen	Zuweisung, WENN … DANN … SONST … ENDE, SOLANGE … ENDE, ZEIGE (Ausgabe), LIES (Eingabe)
Nicht enthalten	Funktionen mit Rückgabe, Felder, Zeichenketten (bewusst; Ausblick im Text)

```
summe = 0
zahl = 5
SOLANGE zahl > 0
  summe = summe + zahl
  zahl = zahl - 1
ENDE
ZEIGE summe
```

Der Übersetzer zeigt seine Arbeit in vier umschaltbaren Ansichten: **Wörter** (Zerlegen), **Baum** (Ausdrucksbaum), **Befehle** (Zuse-K-Assembler mit Zuordnung zur Quellzeile, gleiche Farbe *und* Zeilennummer) und **Speicher** (fertiges Abbild). Im Einzelschritt wird die Quellzeile neben dem gerade ausgeführten Befehl markiert. Im Wettbewerb »Wer ist kürzer?« vergleicht das Kind sein eigenes Assembler-Programm mit dem des Übersetzers.

5. Lernpfad

Experimentarten wie Zuse 2 (entdecken, bauen, zahlen, programmieren), neu **messen**: zwei Bauweisen mit demselben Programm vergleichen und eine Messfrage beantworten.

Stufe	Leitfrage	Experimente (Art, Ergebnis)
0 Auftakt	Warum ruckelt mein Spiel?	0.1 Der Zuse-8 ist zurück (programmieren: Spiel aus Zuse 2 mit Tastenabfrage) · 0.2 Wie viel Zeit kostet Fragen? (messen: Anteil der Abfrageschleife)
1 Unterbrechungen	Wie meldet sich die Welt beim Rechner?	1.1 Die Klingel am Rechner (bauen: Unterbrechungseingang mit Merker) · 1.2 Wohin zurück? (bauen: Rücksprungadresse automatisch auf den Stapel) · 1.3 Nichts kaputt machen (programmieren: Register retten, RETI) · 1.4 Wer zuerst? (bauen: zwei Quellen mit Rangfolge, EI/DI) · 1.5 Fragen gegen Klingeln (messen)
2 Zeitgeber	Wie misst ein Rechner Zeit, ohne zu warten?	2.1 Warten kostet (messen: Warteschleife zählen) · 2.2 Der Zeitgeber (bauen: Zähler, Vorteiler, Vergleicher) · 2.3 Pünktlich jede Sekunde (programmieren: Zeitgeber-Unterbrechung, Uhr auf der Siebensegmentanzeige) · 2.4 Ein Ton von allein (bauen: Zeitgeber schaltet einen Ausgang, hörbarer Rechteckton)
3 Ports als Speicheradressen	Wie spricht ein Programm mit Geräten?	3.1 Geräte im Speicher (bauen: Adressdecoder blendet Ports ein) · 3.2 Eingang oder Ausgang? (bauen: Richtungsregister je Bit) · 3.3 Ein Bit gezielt ändern (programmieren: Masken mit UND/ODER) · 3.4 IN/OUT gegen Speicheradresse (messen)

Stufe	Leitfrage	Experimente (Art, Ergebnis)
4 Programm und Daten	Darf ein Programm sich selbst verändern?	4.1 Das Programm überschreibt sich (programmieren: Fehler in der Von-Neumann-Bauweise) · 4.2 Zwei Speicher (bauen: Harvard mit zwei Bussen) · 4.3 Holen und Lesen zugleich (messen) · 4.4 Konstanten im Programmspeicher (programmieren)
5 Speicherhierarchie	Warum ist Speicher langsamer als der Prozessor?	5.1 Der langsame Speicher (messen: Wartetakte) · 5.2 Der kleine schnelle Zwischenspeicher (bauen: Cache mit 4 Zeilen) · 5.3 Treffer und Fehlschuss (messen: Schleife gegen Sprünge quer durch den Speicher) · 5.4 Wenn der Speicher sich ändert (bauen: Durchschreiben)
6 Fließband	Kann ein Prozessor mehrere Befehle gleichzeitig bearbeiten?	6.1 Holen, während er rechnet (bauen: Band mit drei Stufen) · 6.2 Ein Sprung leert das Band (messen) · 6.3 Warten auf das Ergebnis (bauen: Datenkonflikt erkennen, Wartetakt) · 6.4 Das schnellere Programm (programmieren: Befehle umstellen)
7 Mehrere Kerne	Werden zwei Kerne doppelt so schnell?	7.1 Zwei Kerne, ein Speicher (bauen: Schiedsrichter am Bus) · 7.2 Die falsche Summe (programmieren: beide zählen dieselbe Variable – Wettlauf) · 7.3 Die Sperre (programmieren: TAS) · 7.4 Arbeit teilen (messen: doppelt so schnell?)
8 Vom Text zum Befehl	Was macht ein Übersetzer aus meinem Text?	8.1 Rechnen in Worten (programmieren: Zuweisung → LDA/ADD/STA) · 8.2 WENN … DANN (Vergleich und Sprung) · 8.3 SOLANGE (Schleife mit Rücksprung) · 8.4 Wer ist kürzer? (messen: eigenes Assembler-Programm gegen den Übersetzer)
9 Alles auf einem Chip	Was steckt in einem Mikrocontroller?	9.1 Der Zuse-K (zahlen: Kern, Zeitgeber, Ports, Unterbrechungen auf einem Chip) · 9.2 Das Reaktionsspiel (Projekt: Zeitgeber, Unterbrechung, Zufall, Siebensegment) · 9.3 Der echte Mikrocontroller (zahlen: Register eines ATmega328P in Zuse-Darstellung, Ausblick MCU Explorer)

Summe: 10 Lernstufen, 38 Experimente, dazu Freies Labor (Zuse-K mit allen Schaltern für Bauart, Cache, Fließband, Kerne) und Vorführung.

Vertiefungsstufen: Die Stufen 5 (Speicherhierarchie) und 7 (Mehrere Kerne) sind mit »Vertiefung« gekennzeichnet (Wort und Symbol). Ohne sie öffnet sich nach Stufe 4 Stufe 6 und nach Stufe 6 Stufe 8; Stufe 9 setzt sie nicht voraus. Die Lehrkraft kann sie über die Aufgabe gezielt öffnen.

6. Technische Architektur

Bereich	Umsetzung
`logic/`	unverändert (inkl. Z aus Zuse 2); Gatterebene für die neuen Bausteine (Unterbrechungslogik, Zeitgeber, Cache-Zeile, Schiedsrichter)
Rechnermodell	Zuse-K auf Registertransfer-Ebene, taktgenau; Bausteine nach bestandener Prüfung; Optionen Bauart, Cache, Fließband und Kerne als Modellparameter; Zähler im Modell, nie geschätzt
Nebenläufigkeit	Simulation deterministisch; Reihenfolge der Buszugriffe aus Schiedsrichter-Regel oder Würfel mit Startwert, damit ein Wettlauf wiederholbar ist
Übersetzer	eigener kleiner Übersetzer Zuse-Text → Zuse-K-Assembler (Qt-frei, C++17) mit Zwischenständen (Wörter, Baum, Befehle) als Daten für die Ansichten; Schlüsselwörter je Sprache aus einer Tabelle
App	Ausgabe `zuse3` derselben Codebasis, App-ID `com.dcsoftwareengineering.zuse3`; Inhalte in `zuse/content/zuse3`
Tests	Befehlssatz- und Zyklustests je Bauweise (gleiches Ergebnis, andere Taktzahl), Cache-Tests gegen Handrechnung, Übersetzertests (Quelltext → erwarteter Speicherinhalt), Smoke-Test aller Musterlösungen

7. Datenschutz, Kindersicherheit, Barrierefreiheit

Wie Zuse 1 und 2: offline, kein Konto, keine Analyse; Dateien nur lokal. Alle Zustände (Treffer, Wartetakt, Unterbrechung, Kern 1/2) mit Wort oder Symbol, nicht nur Farbe. Bedienelemente ≥ 48 dp. DE, FR, EN, LU einschließlich der Zuse-Text-Schlüsselwörter.

8. Anforderungen

ID	Anforderung	Priorität
B1	Zuse-K mit Unterbrechungen, Zeitgeber, Ports, EI/DI/RETI/TAS	MUSS
B2	Bauart Von-Neumann/Harvard umschaltbar	MUSS
B3	Messleiste mit Takten, Wartetakten, Treffern, Leerungen; Vergleich zweier Läufe	MUSS
B4	Fließband mit drei Stufen und Fließbanddiagramm	MUSS
B5	Cache mit 4/8 Zeilen und Cache-Ansicht	SOLL (Vertiefung)
B6	Zwei Kerne mit Schiedsrichter, wiederholbarer Wettlauf	SOLL (Vertiefung)
B7	Zuse-Text mit Übersetzer und vier Ansichten, Schlüsselwörter in vier Sprachen	MUSS
B8	Lernpfad 10 Stufen / 38 Experimente, Vertiefungsstufen abwählbar	MUSS
B9	Übergang zum MCU Explorer (Registerkarte ATmega328P in Zuse-Darstellung)	MUSS
N1	Zuse-K mit allen Optionen ≥ 10 kHz Programmtakt auf dem Referenztablet	MUSS
N2	Offline, keine Datenerhebung	MUSS

9. Risiken und Maßnahmen

Nr.	Risiko	Eintritt	Auswirkung	Maßnahme
R1	Stoffmenge zu groß für einen Band	MITTEL	HOCH	Vertiefungsstufen 5 und 7 abwählbar; jede Stufe für sich abgeschlossen
R2	Cache, Fließband, Kerne bleiben abstrakt	MITTEL	MITTEL	immer messen statt behaupten; gleiche Programme, Vergleich nebeneinander
R3	Zuse-Text wird mit echter Programmiersprache verwechselt	NIEDRIG	MITTEL	klar als Lehrsprache benannt; Ausblick auf C im MCU Explorer
R4	Überschneidung mit dem MCU Explorer	MITTEL	MITTEL	Arbeitsteilung nach Abschnitt 2.3; Zuse bleibt ohne Elektrik und ohne C
R5	Übersetzer aufwendiger als geplant	MITTEL	MITTEL	Sprachumfang Version 1 fest begrenzt; keine Optimierung außer sichtbaren einfachen Schritten
R6	Wettlauf in der Simulation nicht reproduzierbar	NIEDRIG	MITTEL	deterministische Simulation mit Startwert

10. Umsetzungsplan

Phase	Inhalt	Grobschätzung
Z3-0	Machbarkeitstest: Zuse-K auf Registertransfer-Ebene, Fließband und Cache als Modellparameter, zwei Kerne deterministisch, Übersetzer-Kern; Leistung Desktop und Tablet	15–20 PT
Z3-1	Ausgabe `zuse3`: Messleiste, Fließbanddiagramm, Cache-Ansicht, zwei Bedienfelder	20–25 PT
Z3-2	Stufen 0–4: Unterbrechungen, Zeitgeber, Ports, Harvard	30–40 PT
Z3-3	Stufen 5–7: Speicherhierarchie, Fließband, Kerne	30–40 PT
Z3-4	Stufe 8: Zuse-Text, Übersetzer, Ansichten	25–35 PT
Z3-5	Stufe 9, Übersetzungen, Lehrerhandreichung, Erprobung, Paketierung	20–25 PT

Summe Zuse 3: ca. 140–185 PT. Start erst nach Zuse 2; Windows zuerst.

11. Offene Entscheidungen

Entscheidung	Spätestens
Produktzusatz (Vorschlag: »Zuse 3 – Im Inneren des Prozessors«)	vor Veröffentlichung
Name der Lehrsprache (Arbeitsname Zuse-Text) und Abgleich der Schlüsselwörter mit ZUSE NPL	Phase Z3-0
Vertiefungsstufen 5 und 7 abwählbar (Vorschlag) oder Pflicht	Phase Z3-1
Gemeinsame Ansicht »Register als Bedienfeld« mit dem MCU Explorer als geteilte QML-Komponente	vor Phase M2 des MCU Explorers

12. Fazit und Empfehlung

Zuse 3 führt den selbst gebauten Rechner bis an die Bauweise heutiger Prozessoren heran und bleibt dabei bei der Methode der Reihe: selbst bauen, ausprobieren, benennen. Neu ist das Messen. Jede Verbesserung (Unterbrechung, Zeitgeber, Cache, Fließband, zweiter Kern) wird am eigenen Programm in Takten nachgewiesen, auch dort, wo sie nicht hilft. Mit dem Übersetzer von Zuse-Text wird der letzte unsichtbare Schritt sichtbar. Am Ende steht die Frage, die der MCU Explorer beantwortet: **Wie programmiert man einen echten Chip?**

Nächste Schritte: (1) Konzept bestätigen. (2) Zuse 2 umsetzen. (3) Machbarkeitstest Z3-0 am Ende von Zuse 2 (der Zuse-8 ist dann vorhanden).